

Database Web Programming

COSC 304 – Introduction to Database Systems

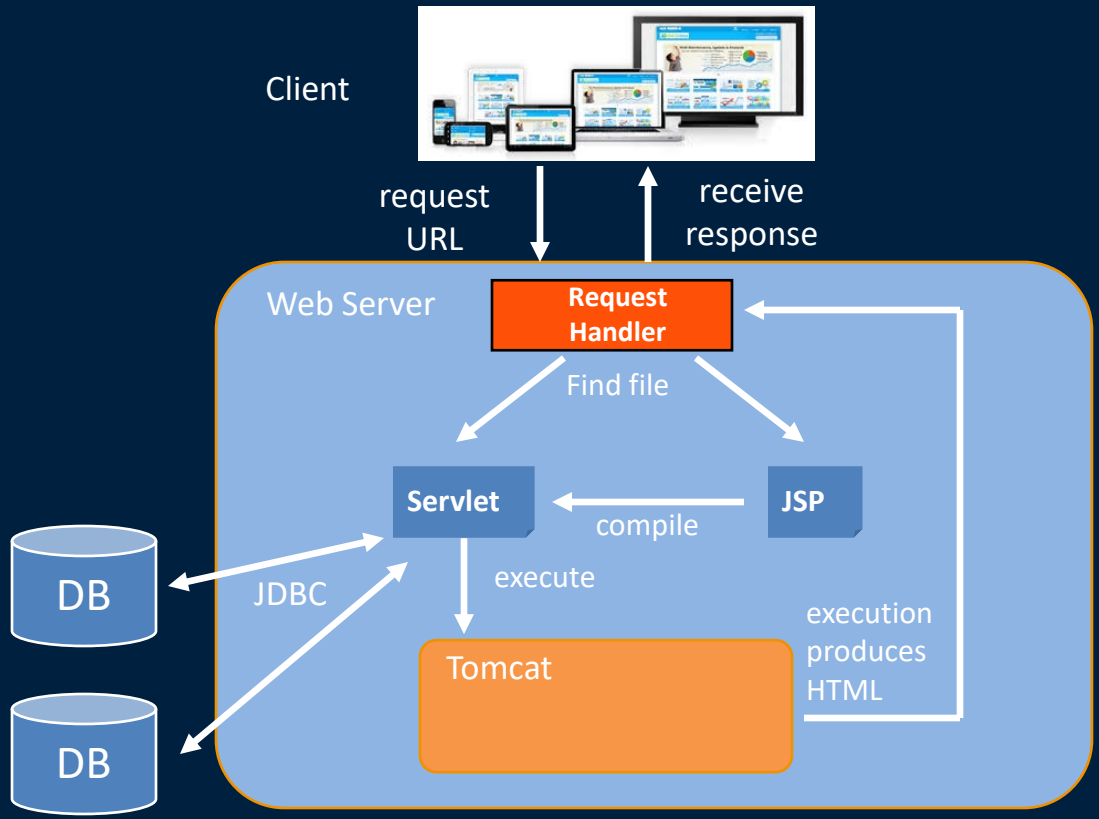
Database Web Programming Overview



The browser does not connect to the database directly.

HTTP is a stateless protocol which requires special handling to remember client state.

Dynamic Web Server Architecture



JSP/Servlet Question

Question: **True or False:** Java JSP/Servlet code runs in the browser.

A) true

B) false

Web Programming Question

Question: **True or False:** JavaScript running in the browser can connect to a database directly using JDBC.

A) true

B) false

Hypertext Markup Language (HTML)

Hypertext Markup Language (***HTML***) describes the layout of a web page including structure, style, images, and links.

An HTML document has ***tags*** in angle brackets **<** and **>** as markup.

- In HTML 5, tags are not case-sensitive. We will use lower case as convention.

Example:

```
<html>
  <head>
    <title>Example HTML with Hyperlink</title>
  </head>
  <body>  <!-- This is a comment-->
    <p>Link: <a href="https://ubc.ca">UBC</a></p>
  </body>
</html>
```

Tables in HTML

Data can be displayed in tables using the **<table></table>** tag.

- Rows are enclosed in table row **<tr></tr>** tags, and each cell is denoted using table data **<td></td>** tags. A header cell uses the **<th></th>** tags.

```
<table border="1">
  <caption>This is my table caption.</caption>
  <tr><th>Tag</th><th>Purpose</th></tr>
  <tr><td>table</td><td>Start and end entire table</td></tr>
  <tr><td>tr</td><td>Start and end each row</td></tr>
  <tr><td>th</td><td>Special column header formatting</td></tr>
  <tr><td>td</td><td>Start and end each data cell</td></tr>
</table>
```

Java Server Pages (JSPs)

Java Server Pages (***JSPs***) provide a layer above `Servlets` that allow you to mix HTML and calls to Java code.

JSPs are converted into `Servlets`, and are easier to work with for those familiar with HTML.

JSP code can use JDBC to connect to a database.

JSP "Hello World!"

```
<html>
<head>
<title>Hello World in JSP</title>
</head>
<body>

<% out.println("Hello World!"); %>

</body>
</html>
```

← Java code embedded in HTML file.

Useful JSP Code

Include another file:

```
<%@ include file="commontop.html" %>
```

Import from the Java library:

```
<%@ page import="java.util.*" %>
```

Declare and use a variable:

```
<%! private int accessCount = 0 %>
<h2>Accesses to page since JSP initialization:
<%= ++accessCount %></h2>
```

Access session and request information:

```
<h2>Current time: <%= new java.util.Date() %></h2>
<h2>Remote Host: <%= request.getRemoteHost()%></h2>
<h2>Session ID: <%= session.getId() %></h2>
```

JSP and JDBC

```
<%@ page import="java.sql.*" %>
<html><head><title>Query Results Using JSP</title></head><body>
<%  // Note: Need to define url, uid, pw
 try
  ( Connection con = DriverManager.getConnection(url, uid, pw);
    Statement stmt = con.createStatement(); )
  {
    String sql = "SELECT ename, salary FROM emp";
    ResultSet rst = stmt.executeQuery(sql);
    out.println("<table><tr><th>Name</th><th>Salary</th></tr>");
    while (rst.next())
    {  out.println("<tr><td>"+rst.getString(1)+"</td>"
                  +"<td>"+rst.getDouble(2)+"</td></tr>");
    }
    out.println("</table>");
  } catch (SQLException ex)     { out.println(ex); }
%></body></html>
```

Answering Queries using HTML Forms

HTML forms allow users to submit requests to a web server.

- The `action` attribute specifies the JSP/Servlet that processes the request.
- The `method` attribute specifies how data is sent (`GET` or `POST`).
- Server reads parameters, queries database, and generates an HTML response.




```
<form method="get"
  action="https://localhost/tomcat/EmpQuery.jsp">
Name:<input type="text" name="empname" size="25">
Dept:<input type="text" name="deptnum" size="5">
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
```

FORM action and type

FORM buttons and input fields

HTTP `GET` and `POST` Methods

`GET` and `POST` allow HTTP clients to communicate with a web server.

`GET`	`POST`
Sends parameters in the URL	Sends parameters in the request body
Commonly used for retrieving/searching data	Commonly used for submitting or changing data
URLs can be bookmarked and shared	Submitted values are not part of the URL's query string
Parameters are visible in the URL	Parameters are still inspectable in browser developer tools

Example `GET` request: `EmpQuery.jsp?empname=Ann&deptnum=D2`

GET vs. POST Question

Question: Select a **true** statement.

A) Data sent using `POST` is passed as part of the URL.

B) A `GET` request with all parameters cannot be bookmarked.

C) Both `GET` and `POST` form parameters can be read using `request.getParameter()`.

D) A `GET` request is required when inserting data into a database.

Reading Parameters

Parameters passed into the JSP are read using the `request` object:

```
String empName = request.getParameter("empname");
String deptNum = request.getParameter("deptnum");
```

Example `GET` request: `EmpQuery.jsp?empname=Ann&deptnum=D2`

`getParameter()` returns a String for either `GET` or `POST`.

It returns `null` if the parameter is missing, or an empty string if the form submits that field blank.

Reading Parameters
Badly Building a Query – SQL Injection!

```
String empName = request.getParameter("empname");
String deptNum = request.getParameter("deptnum");
try
( Connection con = DriverManager.getConnection(url, uid, pw);
  Statement stmt = con.createStatement(); )
{
  String SQLSt="SELECT ename, salary, dno FROM emp WHERE 1=1";
  if (empName != null && !empName.equals(""))
    SQLSt = SQLSt + " and ename LIKE '%" + empName + "%'";

  ResultSet rst = stmt.executeQuery(SQLSt);
  ...
}
```

Building a Query using a PreparedStatement

```
String sql = "SELECT ename, salary, dno FROM emp WHERE 1=1";
boolean hasEmp = empName != null && !empName.isEmpty();
boolean hasDept = deptNum != null && !deptNum.isEmpty();

if (hasEmp) sql += " AND ename LIKE ?";
if (hasDept) sql += " AND dno = ?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    int p = 1;
    if (hasEmp) ps.setString(p++, "%" + empName + "%");
    if (hasDept) ps.setString(p++, deptNum);
    try (ResultSet rs = ps.executeQuery()) {
        // Generate HTML table from results
    }
}
```

Implementing Login and Security

How does a web application remember that a user has logged in?

- User submits a username and password using an HTML form.
- Server validates the credentials (often using a database).
- On successful login, server stores the authenticated user in a session.
- Protected pages check the session before allowing access.
- On logout, the session is invalidated.

```
// After successful login
session.setAttribute("authenticatedUser", username);

// On a protected page
String user = (String) session.getAttribute("authenticatedUser");
if (user == null) {
    response.sendRedirect("login.jsp"); return; }
```

Maintaining State Using Sessions

HTTP is stateless: each request is independent of previous requests.

Request parameters: Pass information to the server with an individual `request` (`GET` or `POST`).

Sessions: Store information on the server that can be accessed across multiple requests from the same browser.

A browser typically sends a session ID in a cookie so the server can identify its session.

Examples: Login information, shopping carts, and user preferences.

Passing Objects Between Pages

sendingPage.jsp

```
<%@ page import="java.util.ArrayList, java.util.Arrays" %>
<html><head><title>Sending Data Page</title></head><body>
<%
  // Populate ArrayList with values
  ArrayList<Integer> ar = new ArrayList<Integer>(
                              Arrays.asList(3, 7, 2, 9, 5));

  // Store arraylist in a session variable
  session.setAttribute("arraydata", ar);
%>
</body></html>
```

Passing Objects Between Pages

receivingPage.jsp

```
<%@ page import="java.util.ArrayList" %>
<html><head><title>Receiving Data Page</title> </head><body>
<%
  ArrayList<Integer> ar = (ArrayList<Integer>)
                          session.getAttribute("arraydata");

  if (ar == null)
    out.println("<h2>No data sent to page.</h2>");
  else
  { out.println("<h2>Received the following array:</h2>");
    for (Integer value : ar)
      out.println(value + "<br>");
  }
%>
</body>
```

Session Question

Question: **True or False:** Data associated with a session remains on the server.

A) true

B) false

Java Servlets

A `Servlet` is a Java class that processes HTTP requests.

- Extends `HttpServlet` and implements `doGet()` and/or `doPost()`.
- Reads input from the `request` object.
- Uses JDBC to query or update the database.
- Generates a response using the response object.

Relationship to JSP:

- JSP is HTML with embedded Java. Servlets are Java classes that generate responses.
- JSP is convenient for presentation. Servlets are convenient for request processing and application logic.
- Tomcat translates JSP pages into Servlets. Both execute on the server and can use JDBC.

Servlets "Hello World!"

```
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet
{
  // doPost() is similar
  public void doGet(HttpServletRequest request, HttpServletResponse response)
          throws ServletException, java.io.IOException
  {
    String name = request.getParameter("name");
    response.setContentType("text/html");
    java.io.PrintWriter out = response.getWriter();
    out.println("<h1>Hello " + name + "</h1>");
   }
}
```

Import Servlet API

Get & Post

Write out HTML for client

JavaScript

JavaScript is a programming language used to make web pages interactive.

- JavaScript in a web page runs in the browser, unlike JSP/Servlet code, which runs on the server.
- Can respond to user actions, modify HTML, and validate form input.
- Can send requests to the server and receive results without reloading the entire page.
- JavaScript is different from Java despite the similar name.

Important: JavaScript executing in the browser does not use JDBC to access a database directly. It communicates with server-side code.

Hello World Example - JavaScript Code

helloWorld.html

```
<html>
<head><title>HelloWorld using JavaScript</title></head>
<body>

<h1>
    <script>
        document.write("Hello, world!");
    </script>
</h1>

</body>
</html>
```

<script> tag indicating code

JavaScript code

document is HTML document
document.write() puts that text into the document at this location

JavaScript versus JSP

JSP/Servlet: Java code executes on the server and generates HTML (which may include JavaScript).

JavaScript: Executes in the browser after the page is received.

JavaScript can modify the page and communicate with server-side code. Database queries are performed on the server, not directly by browser JavaScript.

Key question: If a JSP generates JavaScript code, where does that JavaScript execute?

Hello World written by JSP Code

helloWorld_JS.jsp

```
<html>
<head><title>HelloWorld using JavaScript</title></head>
<body>

<h1>
    <%
        out.println("<script>");
        out.println("document.write(\"Hello, world!\");");
        out.println("</script>");
    %>
</h1>

</body>
</html>
```

Exercise: Explain how this code executes

JavaScript for Data Validation

JavaScript can validate form input in the browser before sending it to the server. This improves responsiveness and avoids unnecessary requests. The server must also validate all input.

```
Enter a number:
<input type="number" id="num">
<button onclick="checkNumber()">Check</button>

<script>
function checkNumber() {
    let num = document.getElementById("num");
    if (!num.validity.valid || num.value === "")
        alert("Enter a valid number.");
    else
        alert("You entered: " + num.value);
}
</script>
```

AJAX — Asynchronous Server Requests

AJAX allows browser JavaScript to request information from a server without reloading the entire page.

Example: Selecting a country updates the province/state options.

Canada Version



US Version



AJAX – Province/State Example

```
async function changeCountry() {
    const country = document.getElementById("country").value;

    const response = await fetch("provinceState.jsp?country=" + country);
    const text = await response.text();

    if (text.trim() === "INVALID") {
        alert("Unable to retrieve provinces/states.");
        return;
    }

    const states = text.trim() === "" ? [] : text.trim().split(",");
    const list = document.getElementById("state");
    list.replaceChildren();

    for (const state of states)
        list.add(new Option(state, state));
}
```

Database Connection Pools

A ***connection pool*** is a collection of reusable database connections managed by a server to efficiently support multiple requests.

- Opening a new database connection for every request is expensive. Connection pooling reduces connection overhead and improves scalability. However, it does not make individual query execution faster.

Example:

```
DataSource ds = ...; // Configured connection pool

try (Connection con = ds.getConnection()) {
    // Execute SQL using JDBC
} // Connection returned to pool
```

Connection Pools Question

Question: **True or False:** A connection pool will speed up the execution time of an individual query (not considering connection time).

A) true

B) false

Conclusion

Web applications separate browser and server processing. HTML forms send user input using HTTP `GET` or `POST`, while server-side code (JSP/Servlets, Python, or Node.js) accesses databases using parameterized queries to prevent SQL injection.

Sessions maintain state across HTTP requests. They allow web applications to remember users and information such as shopping carts, while connection pools efficiently reuse database connections.

JavaScript provides browser-side interactivity. It supports input validation, dynamic page updates, and asynchronous requests (AJAX) to communicate with the server without reloading the entire page.

Objectives

- Explain how browsers, web servers, and databases interact.
- Use HTML forms and JSP to accept input and execute database queries.
- Explain how prepared statements prevent SQL injection.
- Describe how sessions maintain information across requests.
- Distinguish browser JavaScript from server-side code and explain AJAX.
- Explain the purpose of database connection pooling.


UBC
THE UNIVERSITY OF BRITISH COLUMBIA